

Launch Vehicle Guidance Navigation and Control Simulator

Physics Based Six Degree of Freedom Simulation and Statistical Validation

Thesis-style technical report

Prepared from the `launch_vehicle_simulator` project repository
16 September 2026

Scope statement

This report documents a design-stage research simulator. Its results support algorithm development and uncertainty studies; they are not a flight-certification claim.

Abstract

This report presents an end-to-end launch-vehicle guidance, navigation, and control simulator built around a six-degree-of-freedom rigid-body model, an inertial/GPS/barometric sensor suite, nonlinear attitude and navigation estimation, actuator-limited control, and reproducible Monte Carlo analysis. The implementation includes central gravity, atmosphere-dependent aerodynamic forces, variable mass and thrust scheduling, thrust-vector control, sensor biases and latency, asynchronous measurement handling, a multiplicative attitude error representation using modified Rodrigues parameters, and covariance-based statistical diagnostics. The final verification state contains fourteen passing automated tests. A 1,000-run, 600-second, 10 ms campaign produced valid impacts for all runs. Mean GPS NIS was 0.73 and mean barometer NIS was 0.97; median six-state NEES was 4.87, while the mean NEES of 12.2 indicates a smaller population of high-error cases that requires further investigation. The principal contribution is a transparent and testable workflow for connecting physical modeling, closed-loop control, navigation, dispersion analysis, and estimator credibility.

How to read this report

The report separates implemented facts, numerical results, interpretation, and limitations. Equations use the project's conventions: inertial quantities are expressed in the inertial frame unless a body-frame subscript is shown; position and velocity are Cartesian; attitude is represented internally through quaternions and error-state MRPs. The raw CSV files remain in the repository and are catalogued in Appendix A.

Contents

1	Introduction and objectives
2	Physical model
3	Sensors and navigation estimation
4	Guidance and control
5	Monte Carlo methodology
6	Verification and validation
7	Results and interpretation
8	Relevance to established aerospace practice
9	Limitations and next work
10	Reproducibility
	Appendix A Dataset inventory
	Appendix B Figure archive
	References

1 Introduction and objectives

Launch vehicles are strongly coupled dynamic systems. Small changes in thrust, mass properties, aerodynamic coefficients, wind, thrust-line alignment, or center-of-gravity location can alter attitude, acceleration, apogee, and landing location. A nominal trajectory therefore answers only one question: what happens for one assumed vehicle and environment? A design-oriented simulator must also answer how performance changes when those assumptions are dispersed, whether the controller respects hardware limits, and whether the navigation covariance describes the actual estimation error.

The project objective is to provide that connected analysis environment. The simulator is intentionally modular: dynamics generate the truth state, sensors create rate-limited and delayed observations, the estimator propagates and updates an error-state estimate, the controller commands thrust-vector motion, and Monte Carlo orchestration records mission-level outputs and estimator diagnostics.

1.1 Research questions

Question	Evidence produced by the project
How does plant uncertainty affect flight outcome?	Apogee and impact distributions plus sensitivity ranking.
Can the estimator operate with asynchronous and delayed data?	Sensor timing tests, delayed GPS compensation, NEES/NIS traces.
Does the closed loop respect actuator constraints?	Slew and saturation unit test and control implementation.
Is the covariance informative?	NEES for truth-known simulation and NIS for measurement residuals.
Can another engineer reproduce the campaign?	YAML configuration, deterministic seed, CSV outputs, analysis scripts.

1.2 Contribution and boundary

The contribution is systems integration and validation discipline rather than a claim of a new UKF or new launch-vehicle theory. The project makes common GNC elements executable together, exposes the interfaces between them, and records enough information to diagnose failure modes. It is suitable for education, early design trades, estimator/controller development, and preparation for hardware-in-the-loop work.

2 Physical model

The truth model is a six-degree-of-freedom rigid body with translational and rotational motion. The translational state is integrated in Cartesian coordinates. The rotational state uses a quaternion, which is renormalized after numerical integration to prevent drift away from the unit sphere. Vehicle mass decreases during the powered phase according to the motor schedule, and the inertia model transitions between full and dry vehicle values.

2.1 Translational equations

The inertial translational equation is represented in the standard form

$$\dot{\mathbf{r}}_i = \mathbf{v}_i$$

$$\mathbf{m} \cdot \dot{\mathbf{v}}_i = \mathbf{F}_{i,\text{thrust}} + \mathbf{F}_{i,\text{aero}} + \mathbf{F}_{i,\text{gravity}} + \mathbf{F}_{i,\text{other}}$$

with $\dot{\mathbf{r}}_i = \mathbf{v}_i$. Central gravity is modeled as

$$\mathbf{F}_{i,\text{gravity}} = -\mu \mathbf{m} \mathbf{r}_i / \|\mathbf{r}_i\|^3$$

where μ is the gravitational parameter. Aerodynamic force is computed from relative air velocity, density, reference area, drag coefficient, and lift coefficient. Wind is altitude-dependent and is applied through relative velocity rather than as an additive inertial acceleration.

2.2 Rotation and moments

The angular dynamics follow Euler's rigid-body equation

$$\mathbf{I} \dot{\boldsymbol{\omega}}_\beta + \boldsymbol{\omega}_\beta \times (\mathbf{I} \boldsymbol{\omega}_\beta) = \mathbf{M}_\beta$$

Thrust-vector control produces a force direction and, because the nozzle application point is one metre aft of the nominal center of gravity, a moment. Dispersed transverse center-of-gravity offset and thrust misalignment create additional moments. These couplings are important because attitude error changes the thrust direction and therefore changes the translational trajectory.

2.3 Nominal configuration

Parameter	Nominal value
Dry mass	100 kg
Initial propellant mass	50 kg
Reference area	0.01 m ²
Nominal motor	9,000 N from 0–25 s; taper to zero by 30 s
Dry inertia	diag(10, 10, 1) kg·m ²
Full inertia	diag(12, 12, 1.2) kg·m ²
Wind table	0 to 30 km altitude, downrange and crossrange components
Integration campaign	600 s duration, dt = 0.01 s for final campaign

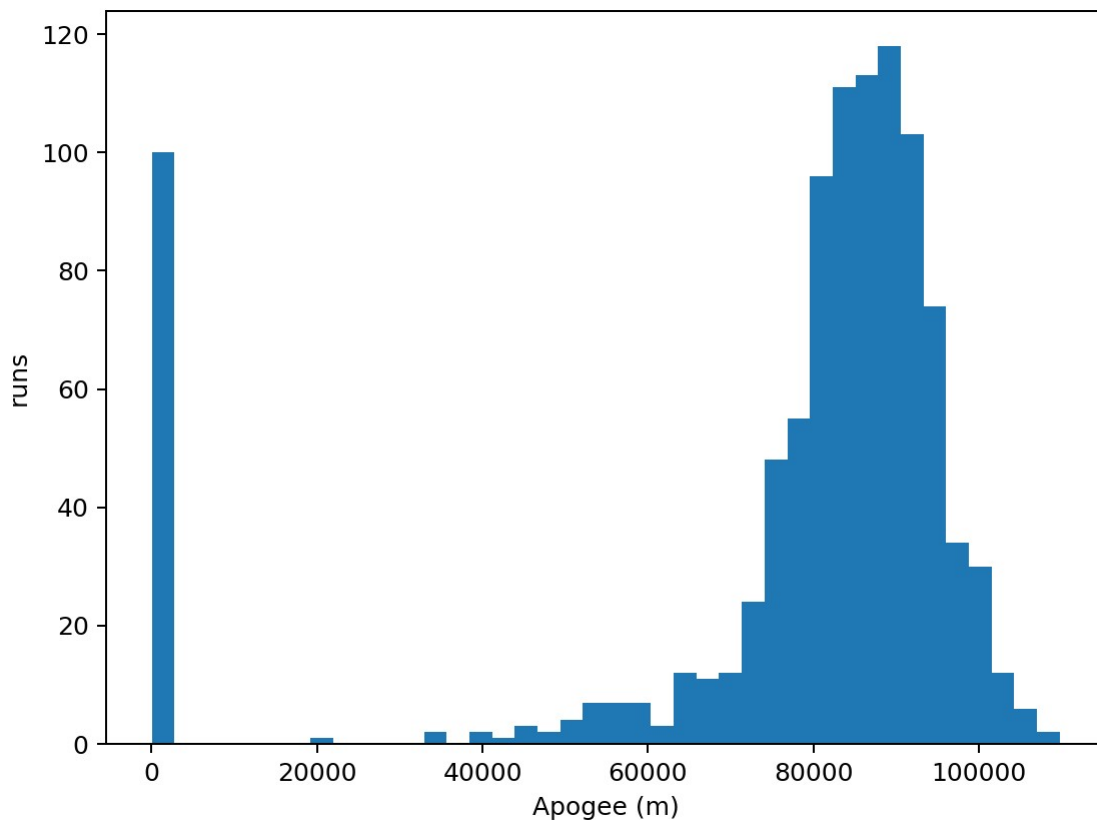


Figure 2.1. Apogee distribution from the final 1,000-run campaign.

3 Sensors and navigation estimation

The navigation architecture is driven by the IMU. Accelerometer specific force and angular-rate samples are consumed at their measurement times, while GPS and barometer measurements arrive through an explicit measurement queue. This distinction matters: the truth state evolves continuously, but measurements are sampled, delayed, and delivered asynchronously.

3.1 Sensor error model

The accelerometer and gyro measurements contain nominal signal, bias, and random measurement error. Turn-on bias and random-walk scale factors are represented in the simulator. GPS position is delivered after an acquisition-

to-arrival delay. The barometer includes a lag and a pressure-to-altitude conversion. These are deliberately explicit model elements so their effects can be tested independently.

3.2 UKF state and propagation

The UKF uses a nonlinear state representation with position, velocity, attitude error, accelerometer bias, and gyro bias. Attitude error is represented with MRPs rather than by averaging quaternions. The MRP shadow set is applied to avoid the 360-degree singularity. During propagation, the latest IMU sample drives the state transition; the powered/coast schedule and propellant depletion remain active in the estimator branch.

A GPS update that arrives after its measurement time is evaluated against the stored state history and then propagated to the current epoch. The covariance update uses Joseph form, which improves numerical symmetry and positive-semidefinite behavior in finite precision.

3.3 Consistency metrics

For a truth-known simulation, the normalized estimation error squared is

$$NEES = \mathbf{e}^T \mathbf{P}^{-1} \mathbf{e}$$

where $\mathbf{e}$ is the state estimation error and $\mathbf{P}$ is the reported covariance. For a measurement update, the normalized innovation squared is

$$NIS = \mathbf{v}^T \mathbf{S}^{-1} \mathbf{v}$$

where $\mathbf{v}$ is the innovation and $\mathbf{S}$ is its innovation covariance. Under the ideal linear-Gaussian assumptions, the expected value is the relevant degrees of freedom. These diagnostics do not prove correctness by themselves, but they reveal overconfidence, excessive conservatism, model mismatch, and outliers.

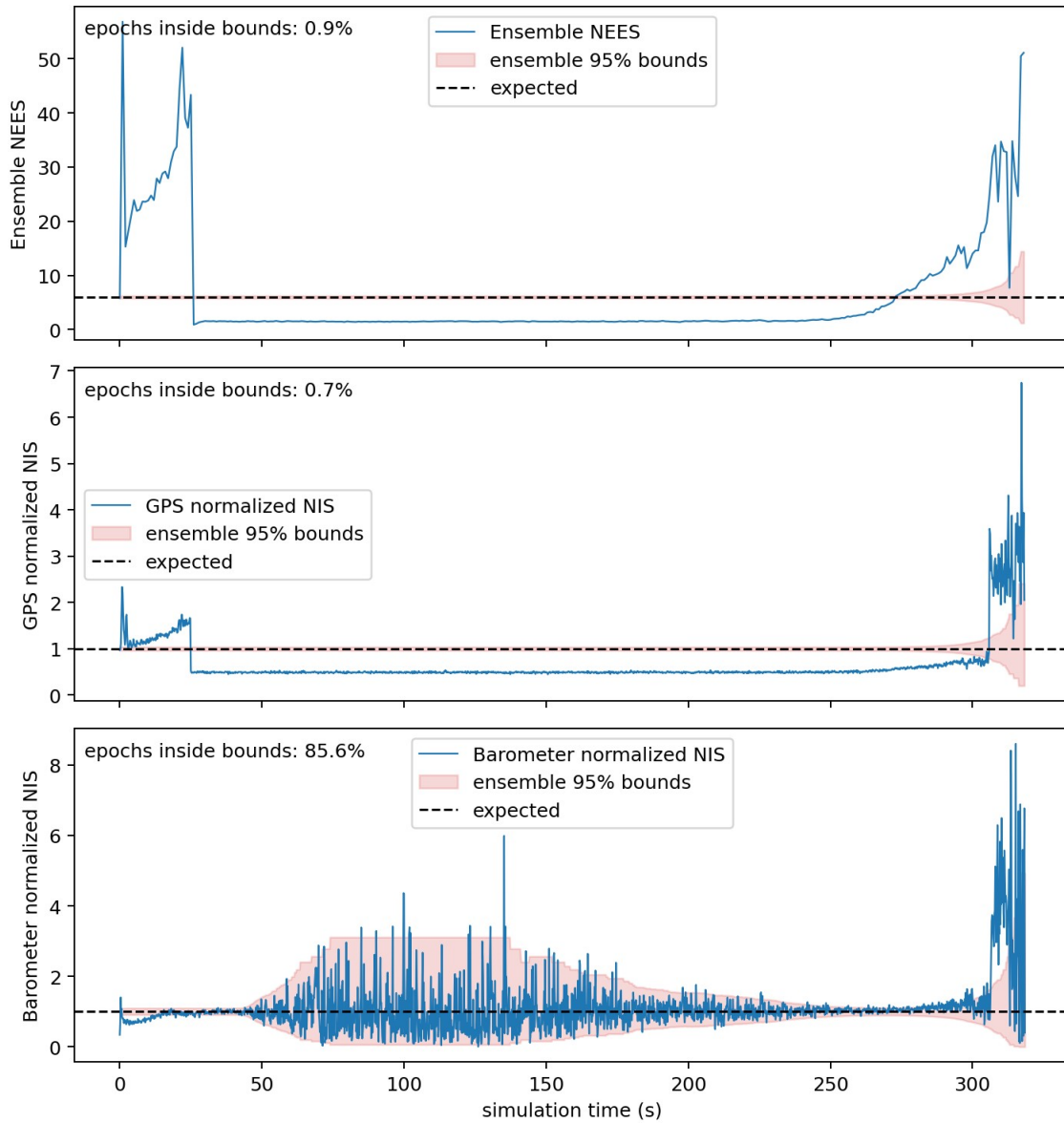


Figure 3.1. Final campaign consistency diagnostics for NEEs, GPS NIS, and barometer NIS.

4 Guidance and control

The controller computes a pointing error from the desired thrust axis and feeds this error together with body angular rate into the attitude-control law. The command is passed through an actuator model with lag, slew-rate limitation, and saturation. This is a critical distinction from an ideal controller: a command that is mathematically correct but physically unattainable can produce misleadingly optimistic results.

4.1 Control implementation checks

Check	Status
Actuator saturation	Verified by automated test
Actuator slew limit	Verified by automated test
Quaternion normalization	Verified by automated test
Thrust-vector moment arm	Implemented through application point
Transverse CG error	Included as a dispersed plant parameter
Closed-loop impact validity	1,000/1,000 in final campaign

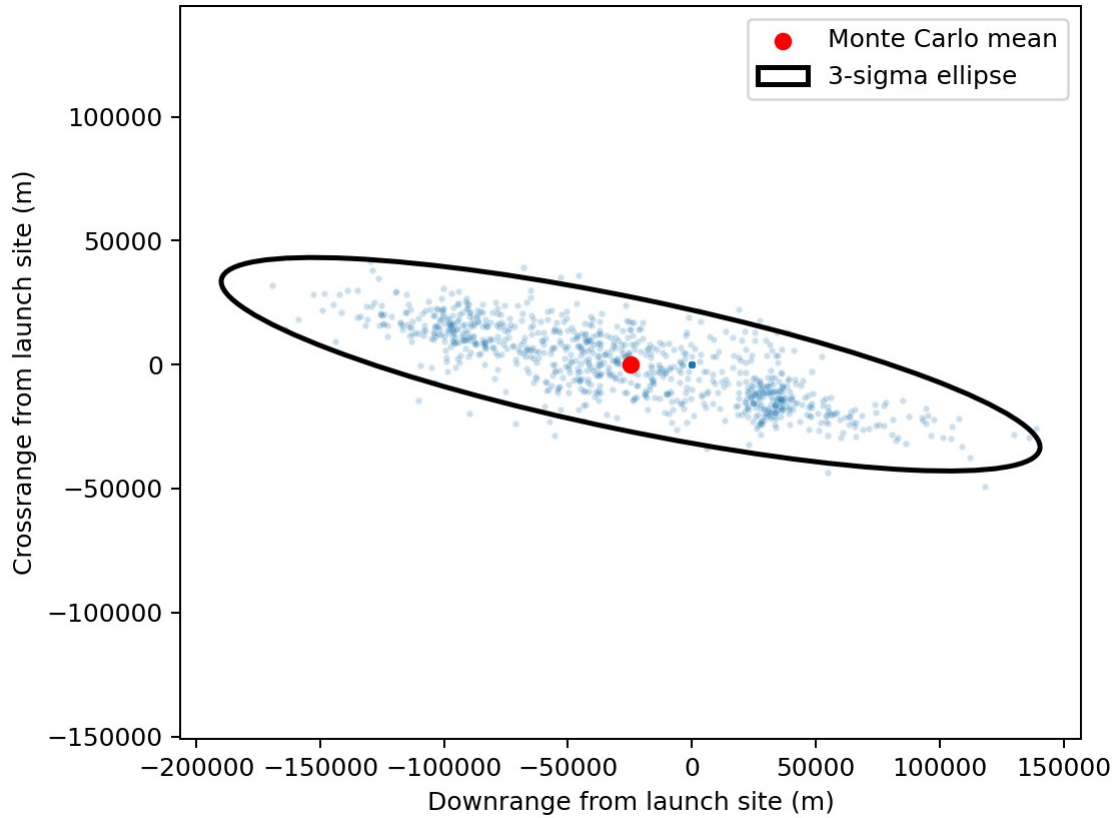


Figure 4.1. Final Monte Carlo landing distribution and three-sigma covariance ellipse.

5 Monte Carlo methodology

Each Monte Carlo run samples independent plant and sensor dispersion variables from the YAML configuration, initializes a deterministic per-run random stream, propagates truth and navigation, closes the control loop, and records mission outputs. The campaign uses OpenMP parallelism, but deterministic run seeding preserves reproducibility independent of thread scheduling.

5.1 Dispersed variables

Variable	Distribution in current YAML	Role
Wind magnitude scale	Normal, $\sigma = 0.20$	Atmospheric disturbance
Thrust scale	Normal, $\sigma = 0.02$	Propulsive performance
Thrust misalignment	Normal, $\sigma = 0.15^\circ$	Thrust-line pointing error
Dry mass scale	Normal, $\sigma = 0.01$	Mass and acceleration
CG location	Normal, $\sigma = 0.001$ m	Thrust moment coupling
Aerodynamic scale	Normal, $\sigma = 0.03$	Force-model uncertainty
Sensor bias scale	Normal, $\sigma = 0.20$	Navigation-model uncertainty

5.2 Outputs

The run-level CSV contains apogee, impact downrange and crossrange, sampled dispersion values, mean consistency metrics, and an impact-valid flag. The trace CSV contains run index, time, metric name, value, and degrees of freedom. Together these files support both mission-level analysis and time-localized diagnosis.

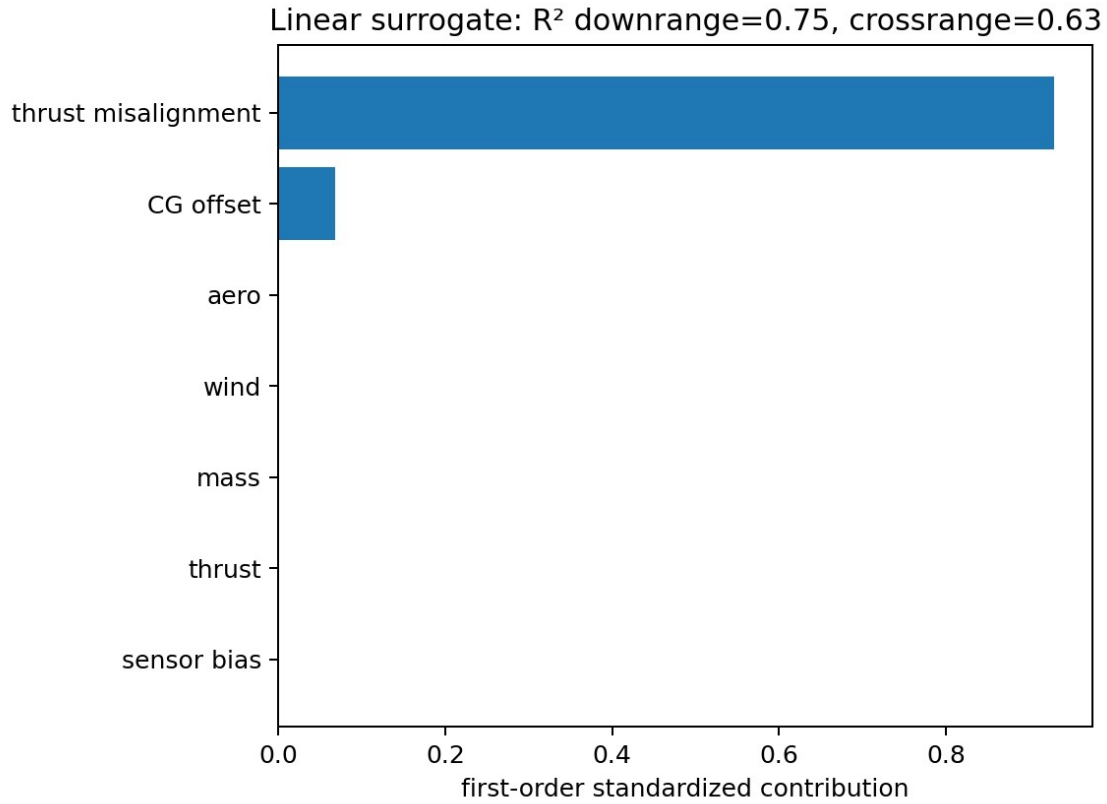


Figure 5.1. Sensitivity ranking from the final analysis pipeline.

6 Verification and validation

Verification asks whether the software implements its intended equations and interfaces. Validation asks whether the model is an adequate representation of the intended physical system. The project currently has strong software verification and an initial statistical validation campaign; it does not yet have hardware or flight correlation.

6.1 Automated verification

Area	Result
Dynamics energy invariant	Pass
Quaternion normalization	Pass
Sensor rates and latency	Pass
Sensor turn-on bias	Pass
Barometer lag timing	Pass
GPS arrival queue	Pass
Actuator slew and saturation	Pass
MRP attitude error handling	Pass
EKF independent propagation/update interface	Pass
Asynchronous IMU rate measurement	Pass
GPS latency compensation	Pass
MRP shadow set	Pass
Linear UKF versus analytic Kalman filter	Pass
Explicit IMU specific-force navigation propagation	Pass

6.2 Numerical campaign

Metric	Final result	Interpretation
Runs	1,000	Adequate for an initial campaign; more may be needed for tail probabilities.
Valid impacts	1,000 / 1,000	No invalid terminal states in the final campaign.
Mean apogee	76,282 m	Mission-level mean under current model and dispersions.
Median apogee	84,983 m	Distribution is skewed by low-apogee cases.
Apogee range	10–109,771 m	Wide outcome spread; investigate low-energy/outlier regimes.
Mean GPS NIS	0.73	Below one; GPS update is conservative on average.
Mean barometer NIS	0.97	Close to one; consistent on average.
Median NEES	4.87 for 6 states	Near expected six-state scale.
Mean NEES	12.2	Elevated by high-error cases; not a blanket pass.

The most important interpretation is the separation between average behavior and tail behavior. GPS and barometer NIS are close to their expected scale, while NEES has a lower median but a higher mean. This pattern is consistent with generally conservative or well-covered estimates punctuated by a smaller set of severe navigation errors. Those cases are valuable targets for further analysis rather than results to hide.

7 Results and interpretation

The final figures show three complementary views. The apogee histogram quantifies vertical-performance dispersion. The landing scatter and ellipse quantify terminal-location uncertainty and the covariance model's geometric summary. The tornado chart ranks which dispersed variables explain the largest normalized contribution to the selected outcome. The consistency figure tests whether the estimator's uncertainty is commensurate with its errors and innovations.

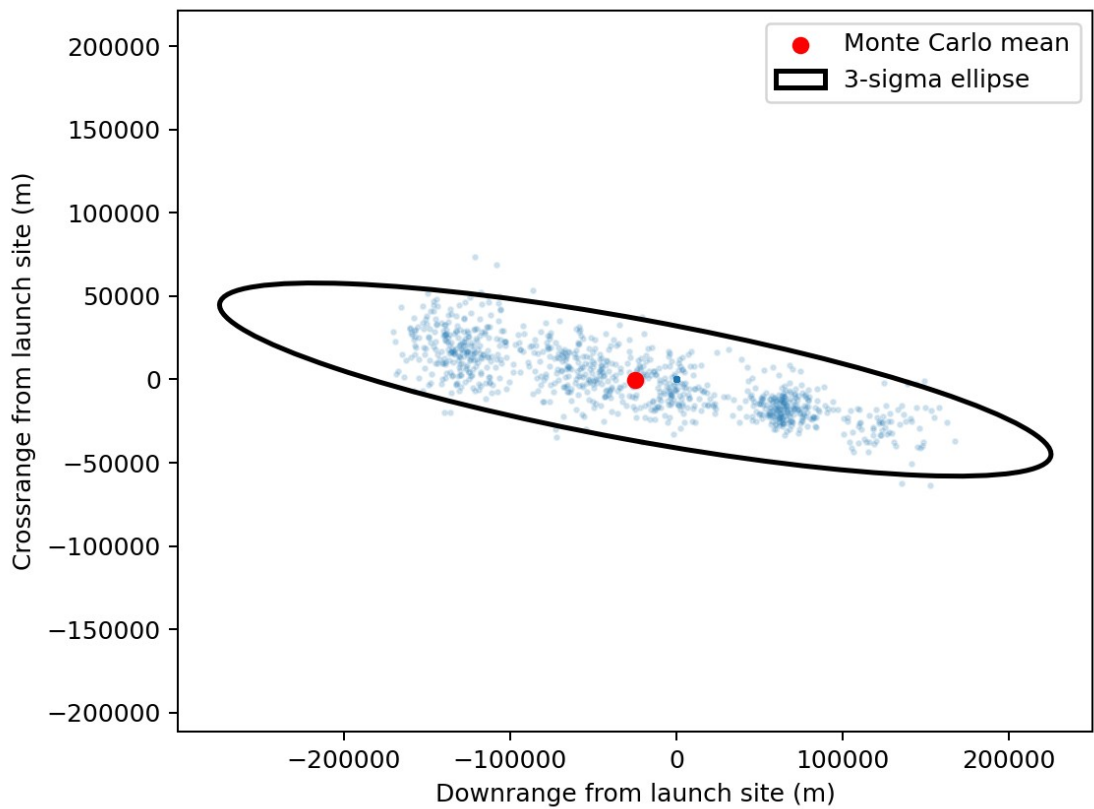


Figure 7.1. Full-campaign landing geometry from the archived full-trace analysis.

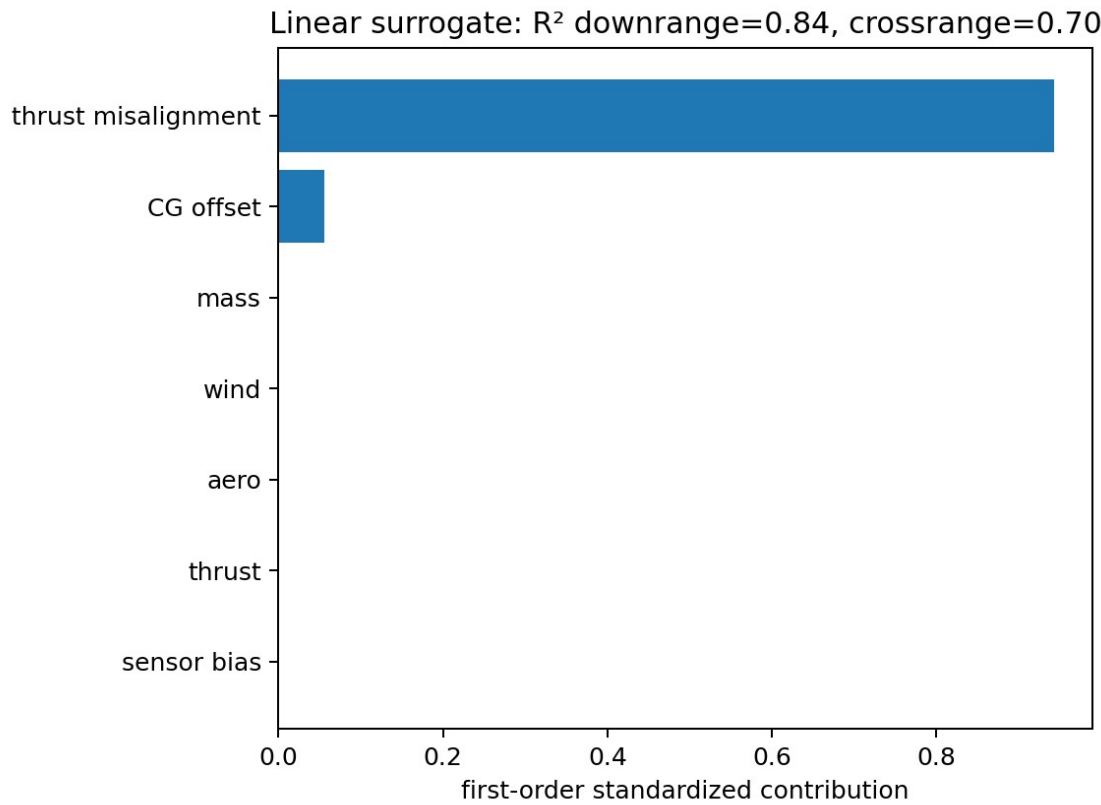


Figure 7.2. Archived full-campaign sensitivity ranking.

7.1 What the results establish

The results establish that the simulator can execute a closed-loop, dispersed, sensor-driven campaign at scale; that the estimator and controller interfaces are exercised by automated tests; that delayed and asynchronous measurements are represented explicitly; and that mission outputs and estimator diagnostics can be analyzed from saved data. They do not establish aerodynamic fidelity, motor-model fidelity, or flight readiness.

7.2 What the results do not establish

The current results cannot be used to claim a certified probability of mission success, a validated landing footprint for a real vehicle, or superiority over a flight-proven navigation system. The current vehicle and atmosphere parameters are representative rather than tied to a documented hardware configuration. OpenRocket/RASAero comparison infrastructure exists in the repository, but a reference export is not yet included, so external trajectory agreement remains unverified.

8 Relevance to established aerospace practice

The project structure is supported by established aerospace practice. NASA documentation describes 6-DOF simulation, Monte Carlo analysis, trajectory simulation, hardware-in-the-loop, and stability analysis as complementary tools in launch-vehicle GNC development. NASA launch-vehicle studies also use dispersed inputs and randomized trajectories to evaluate performance and requirements. A NASA X-43A report describes preflight Monte Carlo analysis as part of achieving robust control-law and guidance performance.

The navigation design is likewise aligned with established integrated-navigation practice. NASA’s INS/GPS tutorial describes inertial propagation corrected by GPS through a Kalman filter. Research on GPS latency identifies delayed delivery as a real issue in dynamic navigation. Consistency literature gives NEES/NIS a specific role: checking whether reported covariance and measurement-noise assumptions agree with realized errors and innovations.

Reference evidence	Direct relevance to this project
NASA SLS 6-DOF cross-verification	Supports modular 6-DOF, Monte Carlo, and verification architecture.
NASA launch-vehicle Monte Carlo requirements analysis	Supports dispersed requirements and probabilistic output analysis.
NASA X-43A 6-DOF Monte Carlo analysis	Supports using randomized preflight analysis to expose robustness limits.
NASA INS/GPS tutorial	Supports IMU propagation with GPS correction.
GPS latency Kalman-filter study	Supports explicit timestamping and delayed-measurement treatment.
Online Kalman consistency tests	Supports NIS/NEES diagnostics and covariance tuning.

9 Limitations and next work

The project is technically credible as a design-stage simulator, but several upgrades are needed before its outputs can support a real vehicle decision.

Priority	Work item	Reason
1	NEES covariance tuning	Reduce powered-phase outliers and separate genuine model mismatch from covariance conservatism.
2	Measured vehicle and motor data	Replace representative coefficients and thrust schedule with traceable data.
3	External trajectory comparison	Run identical initial conditions through OpenRocket or RASAero and document differences.
4	Hardware-in-the-loop sensor timing	Check timestamp, queue, and actuator assumptions against actual interfaces.
5	Larger tail-probability campaign	Use more than 1,000 runs when estimating rare-event probabilities.
6	Convergence and sensitivity studies	Demonstrate timestep and integration-method convergence across the full flight.

A particularly important next experiment is a controlled covariance-tuning matrix. Hold the truth model fixed, vary powered-phase process-noise parameters, and compare NEES mean, median, percentile, and time history. Tuning should be judged jointly with NIS and innovation whiteness; a single average number is insufficient.

10 Reproducibility

The repository contains the source, CMake build, tests, YAML configuration, analysis scripts, and result artifacts. The following commands reproduce the main software checks and a smaller closed-loop run:

```
cd /Users/arvindkc/Downloads/launch_vehicle_simulator
cmake --build build --target dynamics_tests -j1
ctest --test-dir build --output-on-failure
./build/run_monte_carlo /tmp/closed_loop_check.csv . 1 30 0.002 1 1
```

For the formal campaign, use the YAML-controlled run count, duration, timestep, thread count, seed, and dispersion values in config/dispersions.yaml. The run-level and trace-level CSVs listed in Appendix A are the data products used by the plots in this report.

Appendix A Dataset inventory

All datasets below are preserved in `/Users/arvindkc/Downloads/launch_vehicle_simulator/results`. Row counts exclude the header unless noted. The trace datasets are intentionally kept as external CSV companions because they contain millions of time-stamped records.

Dataset	Data rows	Purpose
smoke.csv	20	Basic smoke run outputs
closed_loop_v2.csv	5	Short closed-loop run
transverse_smoke.csv	100	Transverse disturbance smoke run
impact_smoke.csv	100	Impact-validity smoke run
consistency_20.csv	20	20-run consistency campaign
consistency_100.csv	100	100-run consistency campaign
consistency_trace_100.csv	803,111	Trace for 100-run campaign
formal_1000.csv	1,000	Formal 1,000-run campaign
formal_1000_trace.csv	2,774,660	Trace for formal campaign
formal_1000_full.csv	1,000	Archived full-flight campaign
formal_1000_full_trace.csv	6,634,803	Archived full-flight trace
formal_1000_final.csv	1,000	Final tuned 1,000-run campaign
formal_1000_final_trace.csv	6,333,701	Final tuned trace
mc_results.csv	10,000	Original 10,000-run output
mc_results_v2.csv	10,000	Second 10,000-run output
mc_1000.csv	1,000	Earlier 1,000-run output

Run-level schema

Run-level files record: `run`, `apogee_m`, `impact_x_m`, `impact_y_m`, `wind_scale`, `thrust_scale`, `dry_mass_scale`, `aero_scale`, `sensor_bias_scale`, `thrust_misalignment_deg`, `cg_offset_m`, `mean_nees`, `mean_nis`, `mean_gps_nis`, `mean_baro_nis`, and `impact_valid`. Earlier files may omit the separate GPS and barometer NIS columns.

Trace schema

Trace files record: `run`, `time_s`, `metric`, `value`, and `dof`. Metric values include NEES and sensor-specific NIS records. The `dof` column identifies the statistical dimension used for interpretation.

Appendix B Figure archive

The repository contains the following figure families. The final tuned family is the primary result set for this report; the full and smoke families are retained for auditability.

Figure family	Contents
figures/formal_1000_final	Apogee distribution, landing ellipse, NEES/NIS consistency, tornado sensitivity
figures/formal_1000_full	Archived full-flight versions of the same four plots
figures/formal_1000	Earlier formal campaign versions
figures/consistency_100	100-run consistency versions
figures/consistency_smoke	20-run smoke consistency versions
figures/	Current top-level apogee, landing, tornado, and consistency plots

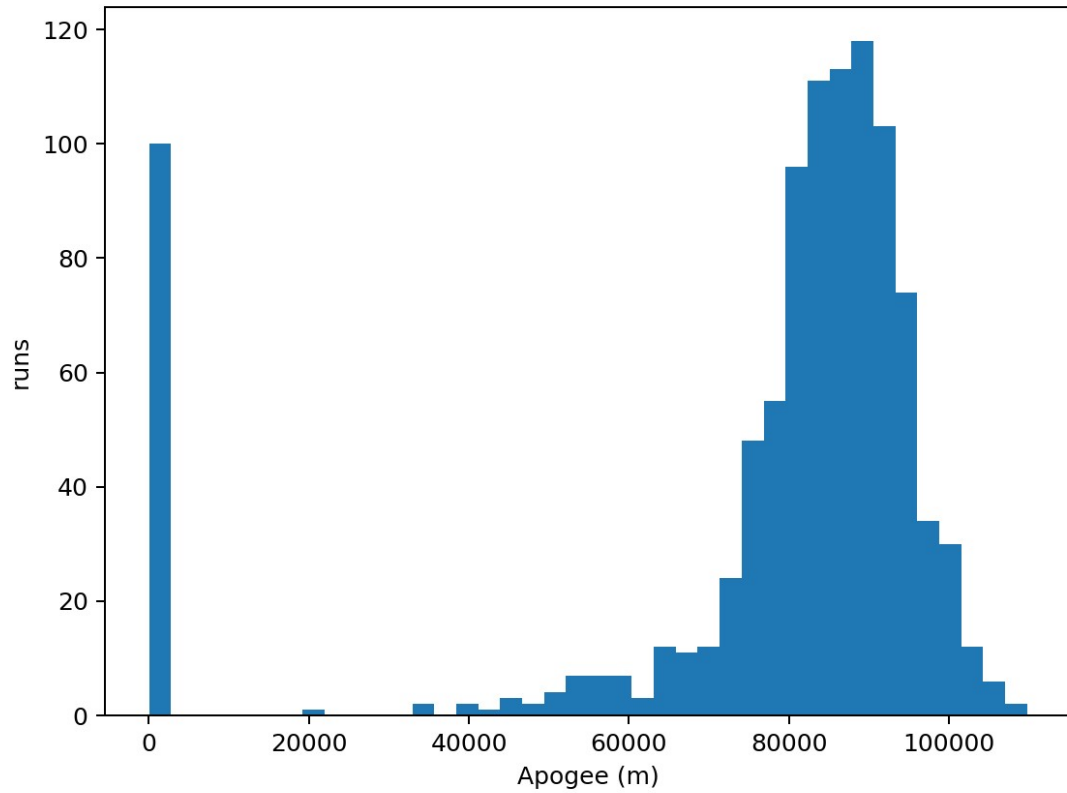


Figure B.1. Primary final apogee distribution.

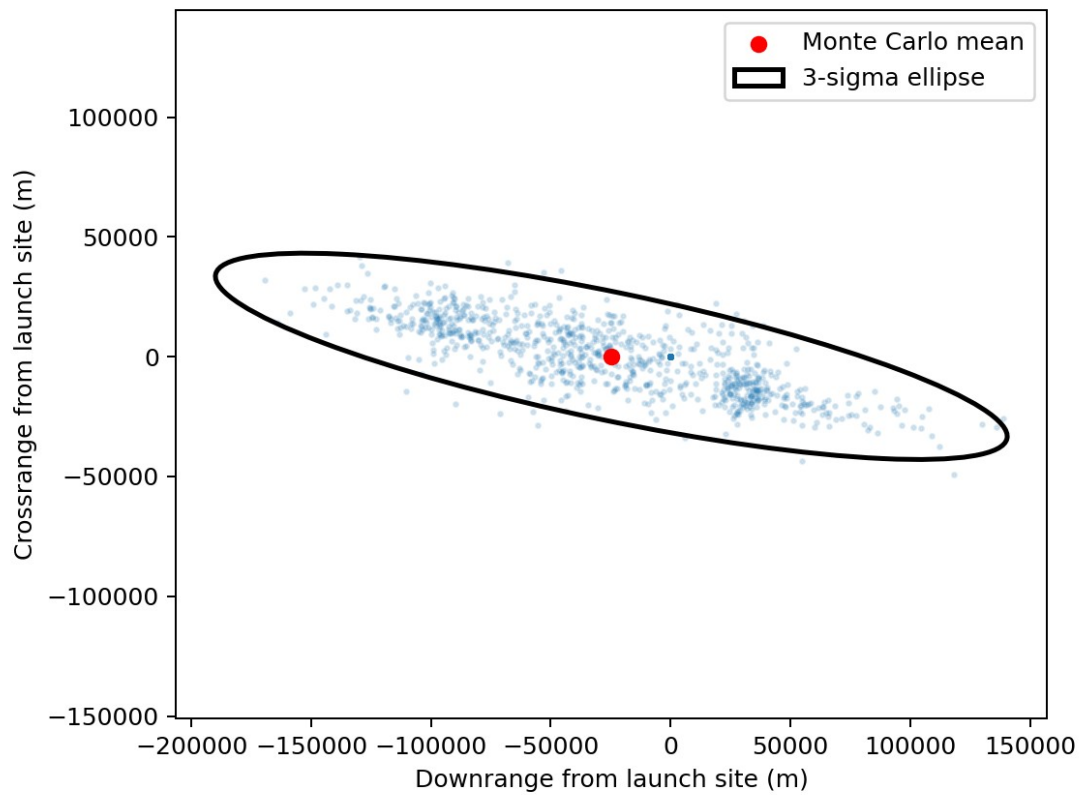


Figure B.2. Primary final landing ellipse.

References

- NASA. Time and Frequency-Domain Cross-Verification of SLS 6DOF Trajectory Simulations. NASA Technical Reports Server. <https://ntrs.nasa.gov/citations/20170001827>
- NASA. Applying Monte Carlo Simulation to Launch Vehicle Design and Requirements Verification. NASA Technical Reports Server. <https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/20100035661.pdf>
- NASA. The X-43A Six Degree of Freedom Monte Carlo Analysis. NASA Technical Reports Server. <https://ntrs.nasa.gov/citations/20080006638>
- Smalling, K. M., and Eure, K. W. A Short Tutorial on Inertial Navigation System and Global Positioning System Integration. NASA/TM-2015-218803. <https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/20150018921.pdf>
- Bouvet, D., and Garcia, G. GPS latency identification by Kalman filtering. Robotica, 18(5), 475–485. <https://doi.org/10.1017/S0263574700002654>
- Piché, R. Online tests of Kalman filter consistency. Tampere University repository. <https://trepo.tuni.fi/bitstream/handle/10024/212922/KFconsistency2015.pdf?sequence=-1>
- Chen, Z., et al. Kalman Filter Auto-tuning through Enforcing Chi-Squared Normalized Error Distributions with Bayesian Optimization. arXiv:2306.07225. <https://arxiv.org/abs/2306.07225>